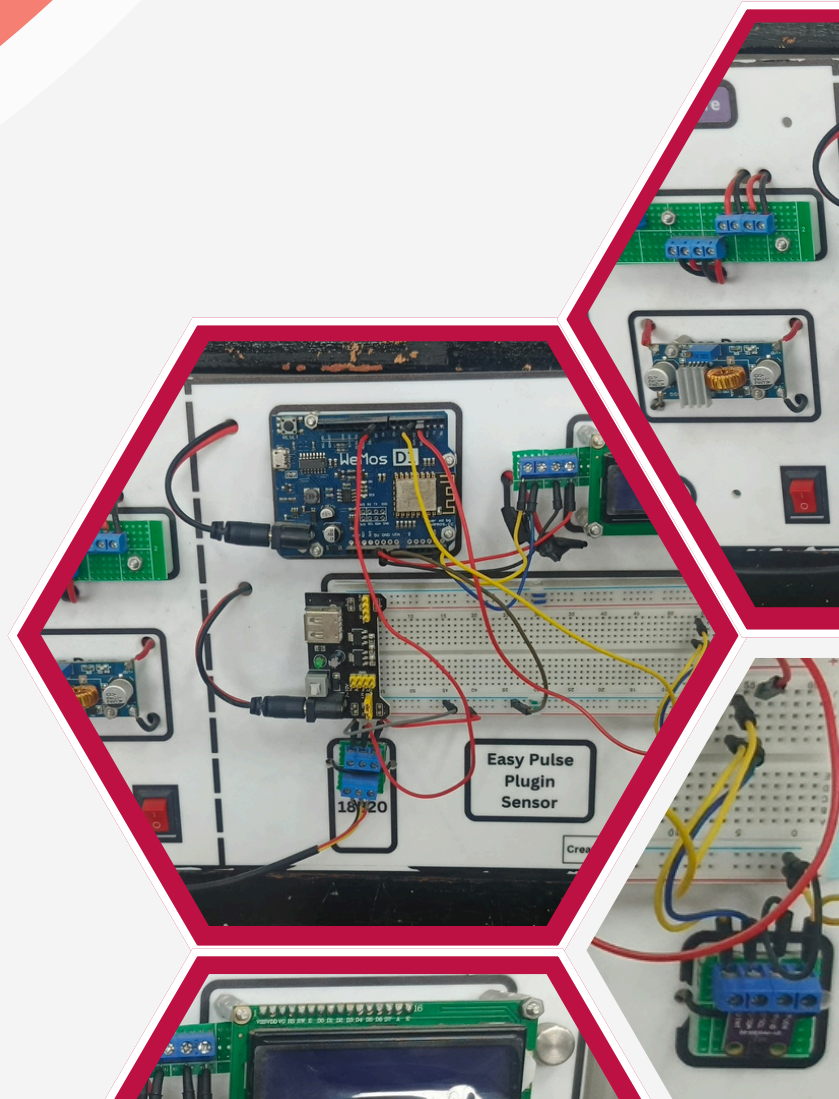


WORKSHOP MODULE

IOT MICROCONTROLLER : HEALTHCARE MONITORING SYSTEM

Objectives :

1. Understanding the Pins Available on the IoT MCU
2. Understanding the Programming of the IoT MCU
3. Learning Functions and Applications of the IoT MCU
4. Understanding the Work of Medical Sensors
5. Understanding the Programming of the IoT Technology



Location :
PSMURO Laboratory D343,
D Campus, Gunadarma University

Workshop Mikrokontroler (Healthcare)

Capaian :

1. Memahami Perangkat Mikrokontroler IoT (Internet of Things)
2. Memahami Pemrograman Mikrokontroler IoT (Internet of Things)
3. Memahami Fungsi & Implementasi Mikrokontroler IoT (Internet of Things)
4. Memahami Cara Kerja Sensor Medis
5. Memahami Pemrograman Teknologi IoT

Alat :

1. WeMos ESP8266
2. Buku / PDF Workshop
3. Trainer Workshop
4. PC / Laptop

Perangkat Lunak :

1. Arduino IDE
2. ThingSpeak

Indeks Workshop Mikrokontroler (Healthcare)

BAB I (Pengenalan Mikrokontroler IoT ESP8266) :

- 1.1.Deskripsi ESP8266
- 1.2.Jenis-Jenis ESP8266
- 1.3.GPIO pada ESP8266

BAB 2 (Pengenalan Pemrograman Mikrokontroler IoT ESP8266) :

- 2.1.Deskripsi Arduino IDE
- 2.2.Struktur Program Arduino IDE

BAB 3 (Pengenalan Fungsi & Implementasi Mikrokontroler IoT ESP8266) :

- 3.1.Pengenalan Perintah Dasar Program Arduino IDE
- 3.2.Operator Kondisi & Operator Logika pada Arduino IDE
- 3.3.Instalasi Board Manager ESP8266 pada Arduino IDE
- 3.4.Implementasi Mikrokontroler ESP8266 dengan Perangkat Keluaran

BAB 4 (Pengenalan Sensor Medis) :

- 4.1.Deskripsi Detak Jantung
- 4.2.Deskripsi Temperatur Tubuh
- 4.3.Deskripsi Sensor DS18B20
- 4.4.Deskripsi Sensor MAX30102
- 4.5.Deskripsi Sensor Easy-Pulse
- 4.6.Pemrograman Embedded System WeMos ESP8266 pada Arduino IDE

BAB 5 (Pengenalan Pemrograman IoT) :

- 5.1.Deskripsi ThingSpeak
- 5.2.Konfigurasi ThingSpeak
- 5.3.Pemrograman Embedded System IoT WeMos ESP8266 pada Arduino IDE

Daftar Isi Workshop Mikrokontroler (Healthcare)

BAB I Pengenalan Mikrokontroler IoT ESP8266.....	1
1.1. Deskripsi ESP8266.....	1
1.2. Jenis-Jenis ESP8266.....	1
1.3. GPIO pada ESP8266.....	2
BAB II Pengenalan Pemrograman Mikrokontroler IoT ESP8266.....	3
2.1. Deskripsi Arduino IDE.....	3
2.2. Struktur Program Arduino IDE.....	3
BAB III Pengenalan Fungsi & Implementasi Mikrokontroler IoT ESP8266.....	5
3.1. Pengenalan Perintah Dasar Program Arduino IDE.....	5
3.2. Operator Kondisi & Operator Logika pada Arduino IDE.....	6
3.3. Instalasi Board Manager ESP8266 pada Arduino IDE.....	6
3.4. Implementasi Mikrokontroler ESP8266 dengan Perangkat Keluaran.....	7
BAB IV Pengenalan Sensor Medis.....	8
4.1. Deskripsi Detak Jantung.....	8
4.2. Deskripsi Temperatur Tubuh.....	8
4.3. Deskripsi Sensor DS18B20.....	8
4.4. Deskripsi Sensor MAX30102.....	9
4.5. Deskripsi Sensor Easy-Pulse.....	10
BAB V Pengenalan Pemrograman Internet of Things (IoT).....	18
5.1. Deskripsi ThingSpeak.....	18
5.2. Konfigurasi ThingSpeak.....	18

BAB I

Pengenalan Mikrokontroler IoT ESP8266

1.1. Deskripsi ESP8266

ESP8266 adalah modul Wi-Fi yang populer dan terjangkau, dirancang untuk menghubungkan perangkat ke jaringan Internet. Dengan ukuran kecil dan konsumsi daya rendah, ESP8266 sering digunakan dalam proyek Internet of Things (IoT) untuk mengontrol dan memantau perangkat secara nirkabel. Modul ini dilengkapi dengan mikrokontroler yang memungkinkan pengembang untuk menjalankan kode langsung di dalamnya, sehingga memudahkan pengembangan aplikasi berbasis web dan konektivitas cloud.

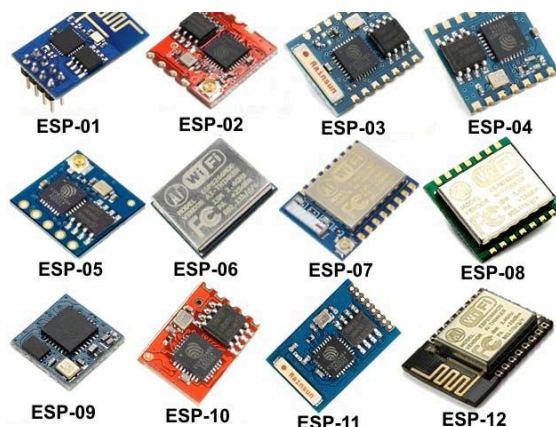


Gambar 1.1 IC ESP8266 (ESP8266MOD AI-Thinker)

1.2. Jenis-Jenis ESP8266

ESP8266 tersedia dalam beberapa varian yang masing-masing memiliki keunggulan dan fitur tersendiri. Salah satu yang paling populer adalah ESP-01, yang memiliki ukuran kecil dan jumlah pin I/O terbatas, ideal untuk proyek sederhana. Variasi lain termasuk ESP-12 dan ESP-12E, yang menawarkan lebih banyak pin I/O, kapasitas memori yang lebih besar, dan kemampuan yang lebih baik untuk aplikasi yang lebih kompleks. Selain itu, ada ESP-07 dan ESP-12F, yang dilengkapi dengan antenna tambahan untuk meningkatkan jangkauan sinyal WiFi. Beberapa modul juga hadir dalam bentuk papan pengembangan seperti NodeMCU dan WeMos D1 Mini, yang menyediakan konektivitas USB dan dukungan pemrograman yang lebih mudah, memungkinkan para pengembang untuk lebih cepat membuat prototipe dan menerapkan proyek IoT.

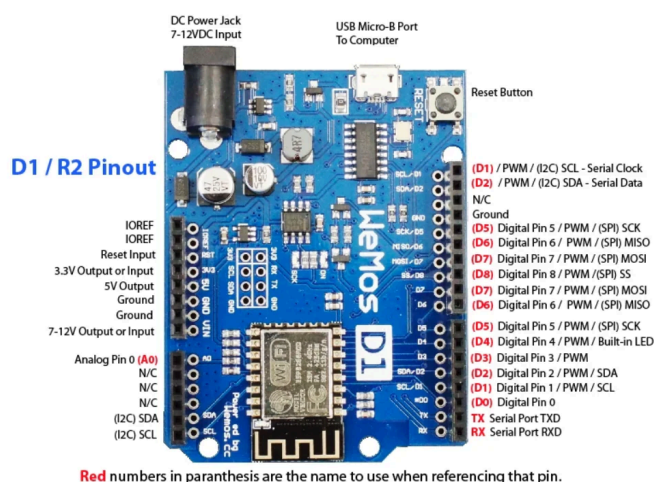
HEALTHCARE (INTERNET OF THINGS)



Gambar 1.2 Macam-Macam IC ESP8266

1.3. GPIO pada ESP8266

General Purpose Input/Output (GPIO) pada ESP8266 adalah antarmuka yang memungkinkan pengguna untuk menghubungkan dan mengendalikan berbagai perangkat eksternal, seperti sensor, aktuator, dan modul lainnya. ESP8266 memiliki total 17 pin GPIO, di mana setiap pin dapat dikonfigurasi untuk berfungsi sebagai input atau output digital, serta beberapa di antaranya mendukung fungsi tambahan seperti PWM (Pulse Width Modulation) dan I2C. GPIO ini memungkinkan pengguna untuk membaca sinyal dari sensor atau mengirimkan sinyal untuk mengontrol perangkat lain, memberikan fleksibilitas tinggi dalam pengembangan aplikasi IoT. Namun, penting untuk diingat bahwa beberapa pin memiliki fungsi khusus, seperti boot dan flash, yang perlu diperhatikan saat merancang sirkuit. Dengan kemampuan ini, GPIO pada ESP8266 menjadikannya pilihan ideal untuk berbagai proyek elektronik dan otomatisasi.



Gambar 1.3 Pinout Development Board ESP8266 dengan Model WeMos D1 R2

BAB II

Pengenalan Pemrograman Mikrokontroler IoT ESP8266

2.1. Deskripsi Arduino IDE

Arduino IDE (Integrated Development Environment) adalah platform perangkat lunak yang digunakan untuk menulis, mengedit, dan mengunggah kode pemrograman ke papan pengembangan Arduino dan mikrokontroler lainnya, termasuk ESP8266. Antarmuka yang ramah pengguna memungkinkan pemula dan pengembang berpengalaman untuk dengan mudah menulis sketsa dalam bahasa pemrograman C/C++, dengan dukungan untuk berbagai pustaka yang memudahkan penggunaan sensor, motor, dan perangkat lainnya. Arduino IDE dilengkapi dengan fitur seperti pengelola pustaka, pemrograman serial, dan pemrograman langsung, serta menyediakan kemampuan debugging dasar.



Gambar 2.1 Contoh Sebuah Program .INO Arduino IDE

2.2. Struktur Program Arduino IDE

Struktur program dalam Arduino IDE umumnya terdiri dari beberapa bagian utama yang saling berinteraksi. Berikut adalah penjelasan tentang struktur program dasar dalam Arduino IDE :

1. Deklarasi Library

Bagian ini digunakan untuk mendeklarasikan library untuk mempersingkat program sehingga memudahkan developer. Selanjutnya objek dari kelas library digunakan untuk menggunakan metode atau fungsi yang disediakan library tersebut.

**HEALTHCARE
(INTERNET OF THINGS)****2. Deklarasi Variabel**

Bagian ini digunakan untuk mendeklarasikan variabel yang akan digunakan di seluruh program. Variabel dapat menyimpan data seperti nilai numerik, string, atau objek.

3. Fungsi Setup

Fungsi ini dijalankan satu kali saat perangkat dihidupkan atau di-reset. Ini adalah tempat untuk inisialisasi yang diperlukan sebelum sistem mulai berjalan.

4. Fungsi Loop

Fungsi ini berisi kode utama yang akan dijalankan berulang kali setelah setup() selesai. Semua logika program seharusnya diletakkan di sini. Fungsi ini akan berhenti dilakukan ketika mikrokontroler dimatikan.

```
#include <Adafruit_Sensor.h>
#include <Adafruit_BMP280.h>

Adafruit_BMP280 bmp; // Membuat objek BMP280

void setup() {
  Serial.begin(9600);
  if (!bmp.begin(0x76)) { // Memulai sensor BMP280
    Serial.println("Tidak dapat menemukan sensor BMP280!");
    while (1);
  }
}

void loop() {
  float suhu = bmp.readTemperature(); // Membaca suhu dari sensor
  Serial.print("Suhu: ");
  Serial.println(suhu);
  delay(2000);
}
```

Gambar 2.2 Contoh Sebuah Program .INO Arduino IDE

BAB III

Pengenalannya Fungsi & Implementasi Mikrokontroler IoT ESP8266

3.1. Pengenalannya Perintah Dasar Program Arduino IDE

Berikut adalah beberapa macam perintah dasar yang umum digunakan dalam Arduino IDE:

1. Fungsi (Input & Output) : Digunakan untuk mendeklarasikan cara kerja sebuah pin GPIO
2. Kondisi (If, If-Else, Switch Case) : Digunakan untuk menjalankan blok kode tertentu hanya jika kondisi yang ditentukan benar
3. Perulangan (For, While) : Digunakan untuk menjalankan blok kode secara terus menerus berdasarkan status / jumlah status tertentu
4. Delay : Digunakan untuk menunda eksekusi program selama waktu yang ditentukan dalam milidetik
5. Komunikasi Serial : Digunakan untuk memulai komunikasi dengan PC / komponen lainnya melalui port / pin serial

Tabel 3.1 Operator Logika pada Arduino IDE

Pin	Syntax	Keterangan
Fungsi I/O	pinMode	Fungsi ini digunakan untuk mengatur mode pin (input atau output)
	digitalRead	Fungsi ini digunakan untuk membaca status dari pin digital yang diatur sebagai input
	digitalWrite	Fungsi ini digunakan untuk mengubah status pin digital (HIGH atau LOW)
	analogRead	Fungsi ini digunakan untuk membaca nilai analog dari pin yang diatur sebagai input
	analogWrite	Fungsi ini digunakan untuk mengatur nilai PWM (Pulse Width Modulation) pada pin yang diatur sebagai output
Kondisi	if...else...	Pernyataan ini digunakan untuk membuat keputusan dalam program
	switch...case	
Perulangan	for...	Digunakan untuk menjalankan blok kode berulang kali dengan kontrol iterasi yang ditentukan
	while...	Digunakan untuk menjalankan blok kode selama kondisi tertentu terpenuhi
Delay	delay	Fungsi ini digunakan untuk menunda eksekusi program selama jumlah milidetik yang ditentukan
Komunikasi Serial	Serial.begin	Fungsi ini digunakan untuk memulai komunikasi serial pada baud rate yang ditentukan
	Serial.print	Fungsi ini digunakan untuk mengirim data ke port serial
	Serial.println	

3.2. Operator Kondisi & Operator Logika pada Arduino IDE

Dalam Arduino IDE, operator kondisi dan operator logika adalah komponen penting yang digunakan untuk mengontrol alur program dan mengambil keputusan berdasarkan kondisi tertentu.

Tabel 3.2 Operator Kondisi pada Arduino IDE

Operator	Deskripsi	Contoh
"=="	Sama Dengan	if (a == b)
"!="	Tidak Sama Dengan	if (a != b)
">"	Lebih Besar Dari	if (a > b)
"<"	Lebih Kecil Dari	if (a < b)
">="	Lebih Besar Sama Dengan	if (a >= b)
"<="	Lebih Kecil Sama Dengan	if (a <= b)

Tabel 3.3 Operator Logika pada Arduino IDE

Operator	Deskripsi	Contoh
"&&"	Operator AND	if (a > 0 && b > 0)
"!"	Operator NOT	if (!(a > 0))

3.3. Instalasi Board Manager ESP8266 pada Arduino IDE

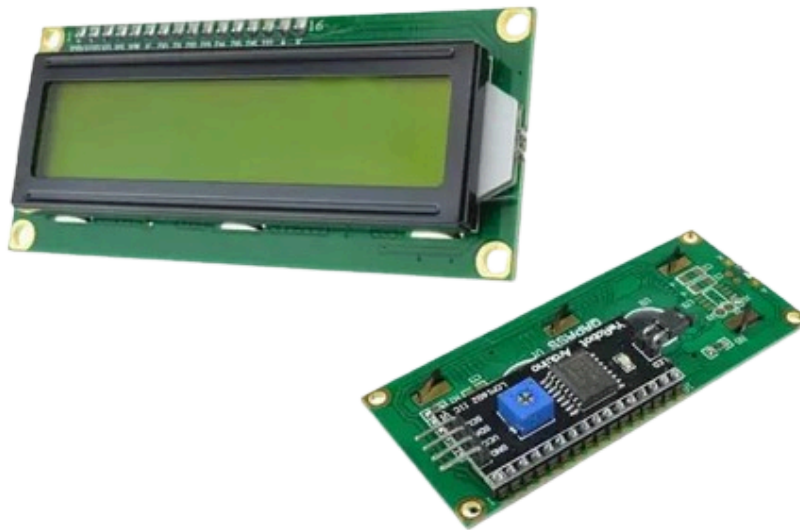
Untuk dapat melakukan pemrograman ESP8266 pada Arduino IDE, terlebih dahulu menginstall board manager untuk ESP8266 sehingga PC / Laptop dapat mendeteksi ESP8266 dengan cara sebagai berikut :

1. Buka Arduino IDE
2. Buka Preferences (File > Preferences)
3. Tambahkan URL
["http://arduino.esp8266.com/stable/package_esp8266com_index.json"](http://arduino.esp8266.com/stable/package_esp8266com_index.json)
pada Additional Board Manager URLs (Preferences > Additional Board Manager URLs)
4. Buka Board Manager (Board > Board manager)
5. Install ESP8266 (ESP8266 by ESP8266 Community pada kotak pencarian)
6. Pilih tipe model board sesuai board yang akan diprogram (Tools > Board > pilih Board Model)

3.4. Implementasi Mikrokontroler ESP8266 dengan Perangkat Keluaran

1. LCD 1602 + I2C Backpack

LCD 1602 adalah layar liquid crystal display (LCD) berukuran 16x2 yang dapat menampilkan hingga 16 karakter dalam 2 baris, sering digunakan dalam proyek elektronik untuk menampilkan informasi secara visual. Kombinasi dengan I2C backpack memungkinkan komunikasi yang lebih sederhana dan efisien dengan mikrokontroler, seperti Arduino atau ESP8266, menggunakan hanya dua pin data (SDA dan SCL) untuk pengoperasian, dibandingkan dengan banyak pin yang diperlukan untuk koneksi langsung. I2C backpack ini juga menyediakan pengaturan kontras dan backlight yang lebih mudah, serta mengurangi kerumitan wiring, sehingga sangat cocok untuk aplikasi yang memerlukan tampilan informasi seperti sensor data, status sistem, atau menu navigasi.



Gambar 3.1 LCD 1602+ I2C Backpack

Tabel 3.4 Keterangan Pin pada LCD 1602 + I2C Backpack

Pin	Nama	Keterangan
VCC	Nama	Sebagai Sumber Daya (5V)
GND	GND	Sebagai Sumber Daya (0V / GND)
SDA	Serial Data Line	Sebagai Pengirim Sinyal Data
SCL	Serial Clock Line	Sebagai Pengirim Sinyal Clock

BAB IV

Pengenalan Sensor Medis

4.1. Deskripsi Detak Jantung

Detak jantung adalah indikator penting dari kesehatan kardiovaskular, dan dengan kemajuan teknologi Internet of Things (IoT) menggunakan ESP8266, pemantauan detak jantung dapat dilakukan secara nirkabel dan real-time. Menggunakan modul ESP8266, perangkat pemantau detak jantung dapat terhubung ke internet dan mengirimkan data detak jantung secara langsung ke platform berbasis cloud atau aplikasi mobile.

Detak jantung normal pada umumnya sekitar 60-100 kali per menit namun ketika berolahraga tentunya akan berbeda. Pada saat berolahraga, detak jantung normal pada orang dewasa yang berusia 20-35 tahun, yakni berkisar 95 hingga 170 kali per menit, pada usia 35-50 tahun berkisar 85 hingga 155 kali per menit. Sedangkan pada lansia di atas 60 tahun, kecepatan jantung berdetak normal ketika berolahraga berkisar 80 hingga 130 kali per menit.

4.2. Deskripsi Temperatur Tubuh

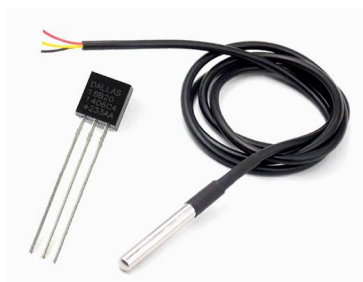
Pemantauan suhu tubuh menggunakan teknologi Internet of Things (IoT) dengan modul ESP8266 menjadi solusi yang semakin populer dalam menjaga kesehatan dan mendeteksi potensi penyakit. Dengan memanfaatkan sensor suhu, seperti sensor DS18B20 atau LM35, perangkat yang dilengkapi ESP8266 dapat mengukur suhu tubuh secara akurat dan mengirimkan data tersebut secara nirkabel ke platform berbasis cloud atau aplikasi mobile.

4.3. Deskripsi Sensor DS18B20

Sensor DS18B20 adalah sensor suhu digital yang populer dan banyak digunakan dalam aplikasi pengukuran suhu, terutama dalam proyek-proyek berbasis microcontroller dan Internet of Things (IoT). Sensor ini menggunakan protokol 1-Wire, yang memungkinkan beberapa sensor untuk dihubungkan pada satu jalur data, sehingga memudahkan instalasi dan pengurangan jumlah kabel yang diperlukan. DS18B20 memiliki rentang pengukuran suhu dari -55°C hingga $+125^{\circ}\text{C}$ dengan akurasi $\pm 0.5^{\circ}\text{C}$,

HEALTHCARE (INTERNET OF THINGS)

menjadikannya ideal untuk berbagai aplikasi, mulai dari pemantauan suhu ruangan hingga pengukuran suhu dalam sistem pendinginan.



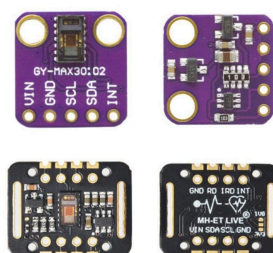
Gambar 4.1 Sensor Temperatur Kontak D18B20

Tabel 4.1 Keterangan Pin pada Sensor D18B20

Pin	Nama	Keterangan
Merah	VCC	Sebagai Sumber Daya (3.3V / 5V)
Hitam	GND	Sebagai Sumber Daya (0V / GND)
Kuning	Data	Sebagai Input Data Sinyal Digital Temperature Kontak

4.4. Deskripsi Sensor MAX30102

Detak jantung adalah indikator penting dari kesehatan kardiovaskular, dan dengan kemajuan teknologi Internet of Things (IoT) menggunakan ESP8266, pemantauan detak jantung dapat dilakukan secara nirkabel dan real-time. Menggunakan modul ESP8266, perangkat pemantau detak jantung dapat terhubung ke internet dan mengirimkan data detak jantung secara langsung ke platform berbasis cloud atau aplikasi mobile.



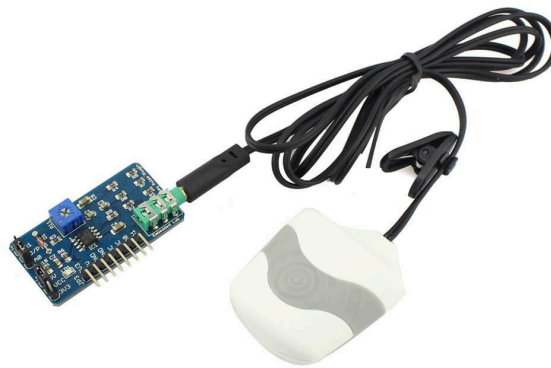
Gambar 4.2 Sensor BPM MAX30102

Tabel 4.2 Keterangan Pin pada Sensor BPM MAX30102

Pin	Nama	Keterangan
VCC	Nama	Sebagai Sumber Daya (5V)
GND	GND	Sebagai Sumber Daya (0V / GND)
SDA	Serial Data Line	Sebagai Pengirim Sinyal Data
SCL	Serial Clock Line	Sebagai Pengirim Sinyal Clock
INT	Interrupt	Sebagai Penghasil Interrupt Pulsa

4.5. Deskripsi Sensor Easy-Pulse

Sensor Easy Pulse adalah sensor yang dirancang untuk mendeteksi detak jantung dengan menggunakan teknologi optik, biasanya melalui prinsip photoplethysmography (PPG). Cara kerja sensor ini melibatkan penggunaan dua LED (sinar merah dan inframerah) dan sebuah photodetector. Ketika sensor ditempatkan pada kulit, LED memancarkan cahaya yang menembus jaringan dan dipantulkan kembali oleh darah yang mengalir di pembuluh darah. Ketika detak jantung terjadi, volume darah dalam pembuluh darah meningkat, menyebabkan perubahan pada intensitas cahaya yang dipantulkan. Photodetector kemudian menangkap cahaya yang dipantulkan dan mengubahnya menjadi sinyal listrik. Sinyal ini kemudian diproses untuk menghitung detak jantung per menit (BPM) dan dapat dikirimkan ke mikrokontroler atau perangkat lain untuk analisis lebih lanjut.



Gambar 4.3 Sensor BPM Easy Pulse

Tabel 4.3 Keterangan Pin pada Sensor Easy Pulse

Pin	Nama	Keterangan
VCC	VCC	Sebagai Sumber Daya (3.3V / 5V)
GND	GND	Sebagai Sumber Daya (0V / GND)
AO	Analog Output	Sebagai Input Data Sinyal Analog untuk Membaca Detak Secara Langsung

Pemrograman Embedded System

WeMos ESP8266 pada Arduino IDE

1. Buatlah program pembacaan Data DS18B20 pada WeMos D1 R2 :

Kode Program
<pre>#include <Wire.h> #include <LiquidCrystal_I2C.h> #include <OneWire.h> #include <DallasTemperature.h> // Data wire is plugged into pin D2 on the WeMos D1 Mini #define ONE_WIRE_BUS D2 // Setup a oneWire instance to communicate with any OneWire devices OneWire oneWire(ONE_WIRE_BUS); // Pass the oneWire reference to DallasTemperature library DallasTemperature sensors(&oneWire); // Set the LCD address to 0x3F for a 16 chars and 2 line display LiquidCrystal_I2C lcd(0x27, 16, 2); void setup(void) { // Start serial communication Serial.begin(115200); Serial.println ("Dallas Temperature Control Library Demo"); // Start up the library sensors.begin(); // Initialize the LCD lcd.begin(); // Turn on the backlight lcd.backlight(); lcd.print(" WELCOME "); lcd.setCursor(0, 1); lcd.print(" TO "); delay(3000); lcd.clear(); lcd.setCursor(0, 0); lcd.print("PSMURO"); lcd.setCursor(0, 1);</pre>

**HEALTHCARE
(INTERNET OF THINGS)**

```
lcd.print("UNIV.GUNADARMA");
delay(5000);
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("MCU HEALTHCARE");
delay(3000);
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Sistem monitor");
lcd.setCursor(0, 1);
lcd.print("Suhu Badan");
delay(5000);
lcd.clear();
lcd.print("Temperature:");
}

void loop(void)
{
  // Request temperature readings
  sensors.requestTemperatures();
  // Read temperature in Celsius
  float tempC =
  sensors.getTempCByIndex(0);
  // Print temperature to serial monitor
  Serial.print("Temperature is: ");
  Serial.print(tempC);
  Serial.println("°C");
  // Print temperature to LCD
  lcd.setCursor(0, 1); // Set cursor to second line
  lcd.print(tempC);
  lcd.print(" C");
  delay(1000);
}
```

**HEALTHCARE
(INTERNET OF THINGS)**

2. Buatlah program pembacaan Data MAX30102 pada WeMos D1 R2 :

Kode Program
<pre>// Include necessary libraries for communication and for the MAX30105 sensor #include <Wire.h> #include "MAX30105.h" #include "heartRate.h" #include <LiquidCrystal_I2C.h> // Create an instance of the MAX30105 class to interact with the sensor MAX30105 particleSensor; LiquidCrystal_I2C lcd(0x27, 16, 2); // Set the LCD address to 0x27 for a 16 chars and 2 line display // Define the size of the rates array for averaging BPM; can be adjusted for smoother results const byte RATE_SIZE = 4; // Increase this for more averaging. 4 is a good starting point. byte rates[RATE_SIZE]; // Array to store heart rate readings for averaging byte rateSpot = 0; // Index for inserting the next heart rate reading into the array long lastBeat = 0; // Timestamp of the last detected beat, used to calculate BPM float beatsPerMinute; // Calculated heart rate in beats per minute int beatAvg; // Average heart rate after processing multiple readings void setup() { Serial.begin(115200); // Start serial communication at 115200 baud rate Serial.println("Initializing..."); lcd.begin(); lcd.backlight(); lcd.setCursor(0, 0); lcd.print(" WELCOME "); lcd.setCursor(0, 1); lcd.print(" TO "); delay(3000); lcd.clear(); lcd.setCursor(0, 0);</pre>

**HEALTHCARE
(INTERNET OF THINGS)**

```
lcd.print("PSMURO");
lcd.setCursor(0, 1);
lcd.print("UNIV.GUNADARMA");
delay(5000);
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("MCU HEALTHCARE");
lcd.setCursor(0, 1);
lcd.print("HEART RATE");
delay(5000);
lcd.clear();
// Attempt to initialize the MAX30105 sensor. Check for a
// successful connection and report.
if (!particleSensor.begin(Wire, I2C_SPEED_FAST))
{
    // Start communication using fast I2C speed
    Serial.println("MAX30102 was not found. Please check
    wiring/power. ");
    while (1); // Infinite loop to halt further execution if
    sensor is not found
}
Serial.println("Place your index finger on the sensor with
steady pressure.");
particleSensor.setup();
// Configure sensor with default settings for heart rate
monitoring

particleSensor.setPulseAmplitudeRed(0x0A);
// Set the red LED pulse amplitude (intensity) to a low value
as an indicator
particleSensor.setPulseAmplitudeGreen(0);
// Turn off the green LED as it's not used here
}

void loop()
{
    long irValue = particleSensor.getIR();
    // Read the infrared value from the sensor
    if (checkForBeat(irValue) == true)
    {
        // Check if a heart beat is detected
        long delta = millis() - lastBeat;
        // Calculate the time between the current and last beat
        lastBeat = millis(); // Update lastBeat to the current time
        beatsPerMinute = 60 / (delta / 1000.0);
        // Calculate BPM
        // Ensure BPM is within a reasonable range before updating
        the rates array
    }
}
```

**HEALTHCARE
(INTERNET OF THINGS)**

```
if (beatsPerMinute < 255 && beatsPerMinute > 20)
{
    rates[rateSpot++] = (byte)beatsPerMinute;
    // Store this reading in the rates array
    rateSpot %= RATE_SIZE;
    // Wrap the rateSpot index to keep it within the bounds
    // of the rates array
    // Compute the average of stored heart rates to smooth out
    // the BPM

    beatAvg = 0;
    for (byte x = 0 ; x < RATE_SIZE ; x++)
        beatAvg += rates[x];
    beatAvg /= RATE_SIZE;
}

Serial.print("IR=");
Serial.print(irValue);
Serial.print(", BPM=");
Serial.print(beatsPerMinute);
Serial.print(", Avg BPM=");
Serial.print(beatAvg);

if (irValue < 50000)
{
    Serial.print(" No finger?");
    lcd.setCursor(0, 0);
    lcd.print("No Finger ? ");
    lcd.setCursor(0, 1);
    lcd.print("Letakan Jari! ");
}
else
{
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("BPM: ");
    lcd.print(beatsPerMinute);
    lcd.setCursor(0, 1);
    lcd.print("Avg BPM: ");
    lcd.print(beatAvg);
}
Serial.println();
//delay(500); // Delay to update the display
//and avoid flickering
}
```

HEALTHCARE
(INTERNET OF THINGS)

3. Buatlah program pembacaan Data Easy Pulse pada WeMos D1 R2 :

Kode Program

```
#include <ESP8266WiFi.h>

int adc;
boolean counter;
int count;
int nilai_bpm;
unsigned long millisBefore;
unsigned long beatTime = 20000;
const int threshold = 500;

void setup()
{
  Serial.begin(115200);
  counter = true;
  millisBefore = millis();
  pinMode(10, OUTPUT);
  digitalWrite(10, HIGH);
  WiFi.begin(ssid, password);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");
}

void loop()
{
  nilai_bpm = (count*(60000/beatTime));
  adc = analogRead(A0);
  //Serial.println(adc);
  delay(10);
  if ((millis() - millisBefore) < beatTime)
  {
    if (counter == true)
    {
      if (adc >= threshold)
      {
        count++;
        counter = false;
      }
    }
  }
}
```

HEALTHCARE (INTERNET OF THINGS)

```

        Serial.print("Beat : ");
        Serial.println(count);
    }
}
if (adc < threshold)
{
    counter = true;
}
}
else
{
    Serial.print(nilai_bpm);
    Serial.println(" BPM");
    count = 0;
    millisBefore = millis();
    delay(4000);
}
}

```

4. Rangkai setiap komponen trainer workshop menyesuaikan tipe pin yang digunakan (Referensi Hal. 2)

Tabel 4.4 Keterangan Interface Pin Embedded System Healthcare

WeMos ESP8266	Protoboard PSU	LCD 1602	Sensor DS1820	Sensor MAX30102	Sensor Easy Pulse
5V	-	VCC	-	-	-
GND	-	GND	-	-	-
D1	-	SCL	-	SCL	-
D2	-	SDA	-	SDA	-
D3	-	-	YELLOW	-	-
A0	-	-	-	-	A0
-	3V3	-	RED	VCC	3V3
-	GND	-	BLACK	GND	5V

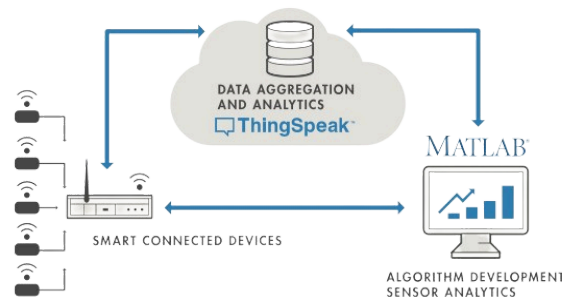
5. Verify & upload program ke dalam ESP8266
(LEPAS JACK DC POWER ESP8266 SEBELUM UPLOAD PROGRAM)

BAB V

Pengenalan Pemrograman Internet of Things (IoT)

5.1. Deskripsi ThingSpeak

ThingSpeak adalah platform layanan cloud yang dirancang khusus untuk Internet of Things (IoT), memungkinkan pengguna untuk mengumpulkan, menganalisis, dan memvisualisasikan data dari perangkat yang terhubung. Dengan antarmuka yang user-friendly, ThingSpeak memfasilitasi pengguna dalam mengirimkan data dari sensor melalui protokol HTTP atau MQTT, serta menyimpan dan memproses data tersebut secara real-time. Platform ini juga menyediakan berbagai alat analisis dan visualisasi, seperti grafik dan peta, yang memudahkan pemantauan dan pengambilan keputusan berbasis data. ThingSpeak mendukung integrasi dengan berbagai perangkat dan aplikasi, serta memungkinkan pengguna untuk menerapkan analisis data lanjutan menggunakan MATLAB. Dengan kemampuannya untuk mendukung proyek IoT secara efisien, ThingSpeak menjadi pilihan populer bagi pengembang, peneliti, dan hobiis dalam mengembangkan solusi pintar yang berbasis data.



Gambar 5.1 Skema ThingSpeak

5.2. Konfigurasi ThingSpeak

Konfigurasi ThingSpeak adalah proses untuk menyiapkan dan mengelola saluran (channel) yang digunakan untuk mengumpulkan dan menganalisis data dari perangkat IoT. Berikut adalah langkah-langkah umum untuk melakukan konfigurasi ThingSpeak :

1. Membuat Akun ThingSpeak

Pertama, kunjungi situs web ThingSpeak dan daftar untuk membuat akun gratis jika Anda belum memilikinya. Setelah mendaftar, masuk ke akun Anda.

**HEALTHCARE
(INTERNET OF THINGS)****2. Membuat Saluran (Channel) Baru**

Setelah masuk, klik pada tab "Channels" dan pilih "My Channels." Kemudian, klik "New Channel" untuk membuat saluran baru.

Berikan nama saluran dan deskripsi yang sesuai. Anda juga dapat menambahkan beberapa field (kolom) untuk menyimpan data, misalnya suhu, kelembapan, atau data lainnya. Setiap field dapat diatur dengan nama yang sesuai dan unit ukur.

3. Mengatur API Keys

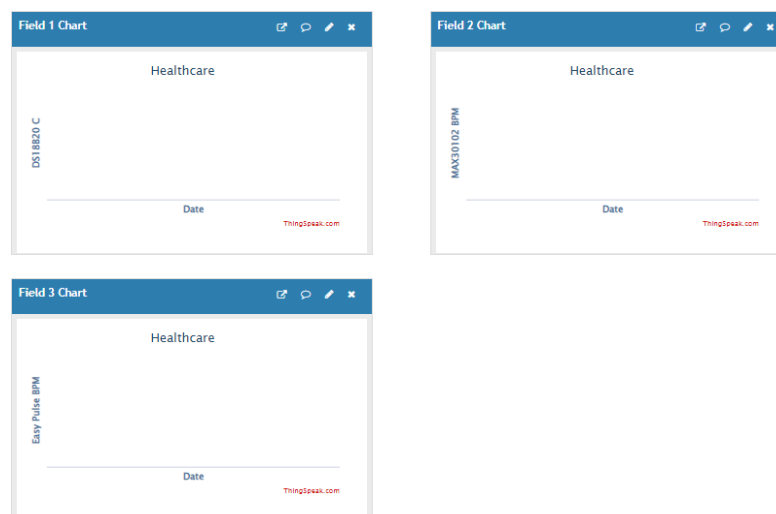
Setelah saluran dibuat, Anda akan melihat kunci API (API Keys) untuk saluran tersebut. Kunci ini digunakan untuk mengirim data dari perangkat Anda ke ThingSpeak. Ada dua kunci yang perlu diperhatikan: Write API Key untuk mengirim data dan Read API Key jika Anda ingin mengambil data dari saluran tersebut.

4. Mengatur Data Format

Anda dapat mengatur format data yang akan diterima, baik dalam bentuk nilai numerik, string, atau data lain sesuai kebutuhan proyek Anda.

5. Visualisasi Data

ThingSpeak juga menyediakan fitur untuk memvisualisasikan data dalam bentuk grafik. Anda dapat mengkonfigurasi tampilan grafik dengan memilih jenis grafik yang diinginkan untuk setiap field, sehingga Anda dapat memantau perubahan data secara real-time.



Gambar 5.2 Tampilan Field (Tempat Menyimpan Data) pada ThingSpeak

Pemrograman Embedded System IoT WeMos ESP8266 pada Arduino IDE

6. Buatlah program Embedded System Healthcare IoT pada WeMos D1 R32 :

Kode Program
<pre> #include <Wire.h> #include "MAX30105.h" #include "heartRate.h" #include <OneWire.h> #include <DallasTemperature.h> #include <ESP8266WiFi.h> #include <ThingSpeak.h> // Define the data pin for the DS18B20 sensor #define ONE_WIRE_BUS D2 // WiFi credentials const char* ssid = ""; const char* password = ""; // ThingSpeak settings unsigned long myChannelNumber = ; const char* myWriteAPIKey = ""; // Create an instance of the MAX30105 class to interact with the sensor MAX30105 particleSensor; OneWire oneWire(ONE_WIRE_BUS); DallasTemperature sensors(&oneWire); // Define the size of the rates array for averaging BPM; can be adjusted for smoother results const byte RATE_SIZE = 4; // Increase this for more averaging. 4 is a good starting point. byte rates[RATE_SIZE]; // Array to store heart rate readings for averaging byte rateSpot = 0; // Index for inserting the next heart rate reading into the array long lastBeat = 0; // Timestamp of the last detected beat, used to calculate BPM float beatsPerMinute; // Calculated heart rate in beats per minute int beatAvg; // Average heart rate after processing multiple readings WiFiClient client; </pre>

**HEALTHCARE
(INTERNET OF THINGS)**

```
void setup()
{
  Serial.begin(115200); // Start serial
communication at 115200 baud rate
  Serial.println("Initializing...");
  // Start up the temperature sensor library
  sensors.begin();
  // Connect to WiFi
  WiFi.begin(ssid, password);
  while (WiFi.status() !=
WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("WiFi connected");
  // Initialize ThingSpeak
  ThingSpeak.begin(client);
  // Attempt to initialize the MAX30105
sensor. Check for a successful connection
and report.
  if (!particleSensor.begin(Wire,
I2C_SPEED_FAST)) { // Start
communication using fast I2C speed
    Serial.println("MAX30102 was not found.
Please check wiring/power.");
    while (1); // Infinite loop to halt further
execution if sensor is not found
  }
  Serial.println("Place your index finger on
the sensor with steady pressure.");
  particleSensor.setup(); // Configure sensor
with default settings for heart rate monitoring

  particleSensor.setPulseAmplitudeRed(0x0A);
// Set the red LED pulse amplitude (intensity)
to a low value as an indicator
  particleSensor.setPulseAmplitudeGreen(0);
// Turn off the green LED as it's not used
here
}

void loop()
{
  long irValue = particleSensor.getIR(); //
Read the infrared value from the sensor
  if (checkForBeat(irValue) == true) { //
Check if a heart beat is detected
```

**HEALTHCARE
(INTERNET OF THINGS)**

```
long delta = millis() - lastBeat; // Calculate
the time between the current and last beat
lastBeat = millis(); // Update lastBeat to
the current time
beatsPerMinute = 60 / (delta / 1000.0); //
Calculate BPM
// Ensure BPM is within a reasonable
range before updating the rates array
if (beatsPerMinute < 255 &&
beatsPerMinute > 20) {
  rates[rateSpot++] =
(byte)beatsPerMinute; // Store this reading in
the rates array
  rateSpot %= RATE_SIZE; // Wrap the
rateSpot index to keep it within the bounds
of the rates array
  // Compute the average of stored heart
rates to smooth out the BPM
  beatAvg = 0;
  for (byte x = 0 ; x < RATE_SIZE ; x++)
    beatAvg += rates[x];
  beatAvg /= RATE_SIZE;
}
// Request temperature readings
sensors.requestTemperatures();
float temperatureC =
sensors.getTempCByIndex(0); // Get
temperature in Celsius
// Output the current IR value, BPM,
averaged BPM, and temperature to the serial
monitor
Serial.print("IR=");
Serial.print(irValue);
Serial.print(", BPM=");
Serial.print(beatsPerMinute);
Serial.print(", Avg BPM=");
Serial.print(beatAvg);
Serial.print(", Temp=");
Serial.print(temperatureC);
Serial.println("°C");
// Send data to ThingSpeak
ThingSpeak.setField(1, beatsPerMinute);
ThingSpeak.setField(2, beatAvg);
ThingSpeak.setField(3, temperatureC);
int x =
ThingSpeak.writeFields(myChannelNumber,
myWriteAPIKey);
```

HEALTHCARE (INTERNET OF THINGS)

```
if(x == 200) {
  Serial.println("Channel update
successful.");
} else {
  Serial.println("Problem updating channel.
HTTP error code " + String(x));
}
// Delay to update the display and avoid
flickering
delay(20000); // ThingSpeak allows updates
every 15 seconds (free accounts)
```

7. Rangkai setiap komponen trainer workshop menyesuaikan tipe pin yang digunakan (Referensi Hal. 2)

Tabel 5.1 Keterangan Interface Pin Embedded System Healthcare

WeMos ESP8266	Protoboard PSU	LCD 1602	Sensor DS1820	Sensor MAX30102	Sensor Easy Pulse
5V	-	VCC	-	-	-
GND	-	GND	-	-	-
D1	-	SCL	-	SCL	-
D2	-	SDA	-	SDA	-
D3	-	-	YELLOW	-	-
A0	-	-	-	-	A0
-	3V3	-	RED	VCC	3V3
-	GND	-	BLACK	GND	5V

8. Verify & upload program ke dalam ESP8266
(LEPAS JACK DC POWER ESP8266 SEBELUM UPLOAD PROGRAM)

